

EEE 416 (January 2025)

Microprocessor & Embedded Systems Laboratory

Final Project Report

Section: A2 Group: 08

Cloud Based Smart Object Locator

Course Instructors:

Ehsanul Karim, Lecturer

Akif Hamid, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Afnan Ali

Student ID: 2006063

Signature: _____

Full Name: Anas Rohan

Student ID: 2006064

Signature: _____

Full Name: Md. Shakib Hossain

Student ID: 2006065

Signature: _____

Full Name: Md. Al Amin Shawon

Student ID: 1806098

Table of Contents

1	Abstract.....	1
2	Introduction	1
3	Design	1
3.1	Problem Formulation (PO(b)).....	1
3.1.1	Identification of Scope	1
3.1.2	Formulation of Problem	2
3.2	Design Method (PO(a)).....	2
3.3	Circuit Diagram	2
3.4	Full Source Code of Firmware.....	3
4	Implementation	10
4.1	Description.....	10
4.1.1	Firebase Realtime Database Setup	10
4.1.2	Web Front-End Design with React	11
4.1.3	ESP32 Hardware Integration	14
4.1.4	LED Animation Programming.....	15
4.1.5	Firebase to ESP32 Communication	17
4.1.6	Physical Box Construction.....	18
4.1.7	OTA (Over-the-Air) Update Integration.....	19
5	Design Analysis and Evaluation	21
5.1	Novelty.....	21
5.2	Design Considerations (PO(c))	21
5.2.1	Considerations to public health and safety	21
5.2.2	Considerations to environment	21
5.2.3	Considerations to cultural and societal needs	21
5.3	Investigations (PO(d)).....	21
5.3.1	Design of Experiment	21
5.3.2	Results and Analysis	21
5.3.3	Interpretation and Conclusions on Data.....	22
5.4	Limitations of Tools (PO(e))	22
5.5	Impact Assessment (PO(f)).....	22
5.5.1	Assessment of Societal and Cultural Issues	22
5.5.2	Assessment of Health and Safety Issues	22

5.5.3	Assessment of Legal Issues	22
5.6	Sustainability Evaluation (PO(g)).....	22
5.7	Ethical Issues (PO(h))	22
6	Reflection on Individual and Team work (PO(i))	23
6.1	Individual Contribution of Each Member	23
6.2	Mode of TeamWork.....	23
6.3	Diversity Statement of Team	23
6.4	Logbook of Project Implementation	24
7	Communication to External Stakeholders (PO(j))	24
7.1	Executive Summary	24
7.2	Github Link.....	24
7.3	YouTube Link.....	25
8	Project Management and Cost Analysis (PO(k)).....	25
8.1	Bill of Materials	25
8.2	Calculation of Per Unit Cost of Prototype	25
8.3	Timeline of Project Implementation	25
9	Future Work (PO(l)).....	26
10	References	27

1 Abstract

In modern electronics labs and workshops, finding specific small components quickly and efficiently is still a major challenge. This project introduces a Cloud-Based Smart Object Locator system that lets users remotely identify and locate components stored in organized boxes. By combining a React-based web interface, Firebase Realtime Database, and an ESP32 microcontroller that controls addressable LEDs, this system provides a simple but effective solution to this issue. Users can choose an item through the web interface, which starts a visual LED animation on the corresponding box. This system also adds a lot of features like object counting, auto reset timer, sign in and logout features. The system also supports OTA (Over-the-Air) updates, allowing remote firmware upgrades without needing physical access. This project shows how cloud communication, embedded control, and web development work together in a real-world IoT system.

2 Introduction

It is not uncommon to have small electronic parts in a lab or workshop. It can be difficult to know where a specific item is stored. Sometimes you need to find an item quickly. Searching through multiple boxes is inefficient. It wastes time and breaks workflow.

For this reason we developed a project called the Cloud-Based Smart Object Locator. It pairs an embedded hardware system with web-based software to improve item location. It allows you to locate a component from anywhere. To find a part you visit a website. We build in the identifier you would use.

The website asks you to select a component. When you do so it lights up an LED animation. It matches the component to the box that contains it. The system is built on an ESP32 microcontroller, Firebase cloud database, and addressable LEDs. The entire item locator is lightweight and wireless. This project also has a lot of features like object counting, auto reset timer, sign in and logout features.

It consumes only minimal power. We also added support for OTA updates. The system can be updated from anywhere without reconnecting. It also doesn't have to be opened to be updated. We hope this example show you how embedded systems and cloud platforms can work together to make a practical system.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The primary goal of this project is to design a cloud-integrated object tracking and indication system. In many laboratory or workshop environments, small components are often misplaced, leading to inefficiencies. The proposed system uses a web interface to allow users to locate items visually using addressable LEDs, controlled by an ESP32

microcontroller, and synchronized via Firebase. This solution supports remote access, is scalable, and ideal for IoT-based applications in academic and industrial environments.

3.1.2 Formulation of Problem

Finding specific electronic components or objects from large mixed inventories is time-consuming. Traditional solutions offer no real-time feedback or user-friendly interface. Problem Statement:

“Design and implement a smart object locator system that uses a cloud-based web interface and an ESP32-controlled LED grid to visually indicate the location of selected objects in real-time.”

This system must be accessible remotely, provide instant feedback, be easily updatable (via OTA), and user-friendly for non-technical users.

3.2 Design Method (PO(a))

This project integrates concepts from electronics, embedded systems, wireless networking, and web development. The design method is broken down as follows:

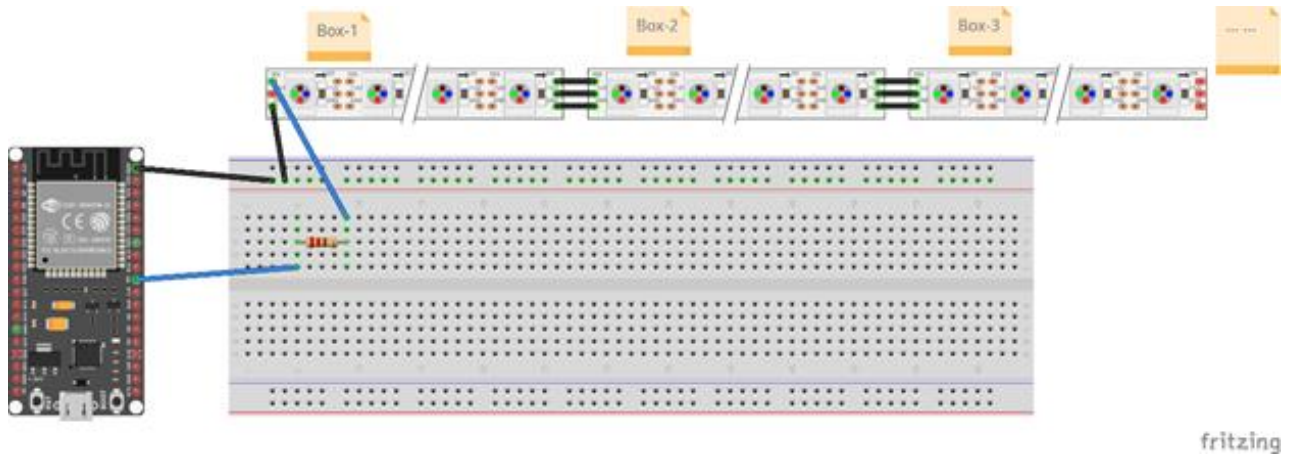
- **Mathematics & Engineering:**
 - Index mapping between database and LED positions
 - Timed animations using modular arithmetic and LED buffer control
- **Electronics:**
 - Circuit built around ESP32 microcontroller
 - WS2812B addressable LEDs wired in series
 - Resistors and capacitors used for filtering and protection
- **Embedded Programming:**
 - ESP32 reads from Firebase using Wi-Fi
 - Animations programmed for each box using C++ logic
 - OTA updates for firmware revisions
- **Software/Web:**
 - React-based frontend for selecting and managing components
 - Firebase Realtime Database for data synchronization
 - Image upload handled via imgbb API

Tools & Platforms Used: - Arduino IDE - Firebase Console - Vercel (for web hosting) - imgbb (image storage)

3.3 Circuit Diagram

- **ESP32 Board**
 - Connected to USB (for power and programming)
 - GPIO pin (GPIO 5) connected to data-in of WS2812B
- **WS2812B LED Strip**
 - Powered via 5V DC source
 - Connected in series (total 200 LEDs, grouped into 10 boxes)

- Ground and Data Line connected appropriately
- **Decoupling Components**
 - 220Ω resistor between ESP32 and LED Data In
 - 1000uF capacitor across 5V and GND to prevent power surge
- **OTA & Wi-Fi Connectivity**
 - ESP32 connects wirelessly to Firebase via Wi-Fi router
 - OTA handled over same network
 -



3.4 Full Source Code of Firmware

```

#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include <NeoPixelBus.h>
#include <ArduinoOTA.h>
#include <ESPmDNS.h>

#define WIFI_SSID "Bliss"
#define WIFI_PASSWORD "abcdefgh"
#define API_KEY "AIzaSyBt6MmNYqLdb2IiPLFR6hS_dSz14bwMjx0"
#define DATABASE_URL "https://draft2-1742d-default-rtdb.asia-southeast1.firebaseio.com/"

// Total number of LEDs (218 LEDs divided into 11 boxes)
const uint16_t NUM_LEDS_TOTAL = 218;
const uint8_t LED_PIN = 4;

NeoPixelBus<NeoGrbFeature, NeoEsp32Rmt0800KbpsMethod> strip(NUM_LEDS_TOTAL, LED_PIN);

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

int lastIndex = -1;
unsigned long lastAnimTime = 0;
int animStep = 0;
float animPhase = 0.0;
bool otaInProgress = false;

void setup() {
  Serial.begin(115200);
  delay(500);

  // Initialize LED strip
  strip.Begin();
  strip.Show();

  // Connect to WiFi with LED feedback
  connectToWiFi();

  // Setup Firebase
  setupFirebase();

  // Setup OTA
  setupOTA();

  Serial.println("✔ Setup complete! Firebase + OTA ready");
}

void connectToWiFi() {
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi");

  // Show WiFi connection progress on first 20 LEDs (Box 1)
  int ledIndex = 0;
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");

    // Light up LEDs one by one while connecting (blue color)
    if (ledIndex < 20) {
      strip.SetPixelColor(ledIndex, RgbColor(0, 0, 255)); // Blue
      strip.Show();
      ledIndex++;
    }

    delay(300);

    // Reset LED index if we've filled box 1 but still connecting
    if (ledIndex >= 20) {
      ledIndex = 0;
      // Clear Box 1
      for (int i = 0; i < 20; i++) {
        strip.SetPixelColor(i, RgbColor(0, 0, 0));
      }
      strip.Show();
      delay(500);
    }
  }

  Serial.println("\nWiFi connected!");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());

  // Clear all LEDs
  strip.ClearTo(RgbColor(0, 0, 0));
  strip.Show();

  // green 3 times to indicate successful WiFi connection

```

```

for(int i = 0; i < 3; i++) {
    for (int j = 0; j < 20; j++) { // Box 1 for status
        strip.SetPixelColor(j, RgbColor(0, 255, 0)); // Green
    }
    strip.Show();
    delay(200);
    for (int j = 0; j < 20; j++) {
        strip.SetPixelColor(j, RgbColor(0, 0, 0)); // Off
    }
    strip.Show();
    delay(200);
}
}

void setupFirebase() {
    config.api_key = API_KEY;
    config.database_url = DATABASE_URL;

    if (Firebase.signUp(&config, &auth, "", "")) {
        Serial.println("Firebase sign-up successful");
    } else {
        Serial.printf("Firebase sign-up failed: %s\n", config.signer.signupError.message.c_str());
        // Flash red to indicate Firebase error
        for(int i = 0; i < 5; i++) {
            for (int j = 0; j < 20; j++) { // Use Box 1 for status
                strip.SetPixelColor(j, RgbColor(255, 0, 0)); // Red
            }
            strip.Show();
            delay(150);
            for (int j = 0; j < 20; j++) {
                strip.SetPixelColor(j, RgbColor(0, 0, 0)); // Off
            }
            strip.Show();
            delay(150);
        }
        while (true);
    }
}

Firebase.begin(&config, &auth);
Firebase.reconnectWiFi(true);
Serial.println("✔ Firebase initialized");
}

void setupOTA() {
    // OTA hostname and password
    ArduinoOTA.setHostname("ESP32-Firebase-LED");
    ArduinoOTA.setPassword("group8"); // Password

    ArduinoOTA.onStart([]() {
        String type = (ArduinoOTA.getCommand() == U_FLASH) ? "sketch" : "filesystem";
        Serial.println("Start updating " + type);

        // Set OTA flag and turn off all LEDs during update
        otaInProgress = true;
        strip.ClearTo(RgbColor(0, 0, 0));
        strip.Show();
    });

    ArduinoOTA.onEnd([]() {
        Serial.println("\nOTA Update completed!");

        // Flash green across all boxes to indicate update
        for(int flash = 0; flash < 3; flash++) {
            strip.ClearTo(RgbColor(0, 255, 0)); // Green
            strip.Show();
            delay(300);
            strip.ClearTo(RgbColor(0, 0, 0)); // Off
            strip.Show();
            delay(300);
        }

        otaInProgress = false;
    });

    ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
        Serial.printf("Progress: %u%%\r", (progress / (total / 100)));

        // Show progress on first 20 LEDs only (Box 1)
        int progressLEDs = map(progress, 0, total, 0, 20);

        // Clear all LEDs
        strip.ClearTo(RgbColor(0, 0, 0));

        // Light up progress LEDs in yellow (Box 1 only)
        for(int i = 0; i < progressLEDs; i++) {

```



```

        strip.SetPixelColor(i, RgbColor(255, 255, 0)); // Yellow
    }

    strip.Show();
});

ArduinoOTA.onError([](ota_error_t error) {
    Serial.printf("Error[%u]: ", error);

    // Flash red across all boxes to indicate error
    for(int i = 0; i < 5; i++) {
        strip.ClearTo(RgbColor(255, 0, 0)); // Red
        strip.Show();
        delay(200);
        strip.ClearTo(RgbColor(0, 0, 0)); // Off
        strip.Show();
        delay(200);
    }

    if (error == OTA_AUTH_ERROR) Serial.println("Auth Failed");
    else if (error == OTA_BEGIN_ERROR) Serial.println("Begin Failed");
    else if (error == OTA_CONNECT_ERROR) Serial.println("Connect Failed");
    else if (error == OTA_RECEIVE_ERROR) Serial.println("Receive Failed");
    else if (error == OTA_END_ERROR) Serial.println("End Failed");

    // Clear OTA flag
    otaInProgress = false;
});

ArduinoOTA.begin();
Serial.println("✔ OTA Ready!");
Serial.println("To update wirelessly:");
Serial.println("1. Go to Tools -> Port in Arduino IDE");
Serial.println("2. Select 'ESP32-Firebase-LED at " + WiFi.localIP().toString() + "'");
Serial.println("3. Upload your code wirelessly!");
}

void loop() {
    // Handle OTA updates
    ArduinoOTA.handle();

    // Check WiFi connection and reconnect if needed
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi connection lost. Reconnecting...");
        connectToWiFi();
        // Re-initialize Firebase after WiFi reconnection
        Firebase.reconnectWiFi(true);
    }

    //
    if (!otaInProgress) {
        handleFirebase();
        animateLEDs();
    }

    delay(10);
}

void handleFirebase() {
    if (!Firebase.ready()) return;

    // Read the selected_index from Firebase (from the root of firebase)
    if (Firebase.RTDB.getString(&fbdo, "/selected_index")) {
        String indexStr = fbdo.stringData(); // Get the value as String
        int index = indexStr.toInt();        // Convert string to integer

        if (index != lastIndex) {
            lastIndex = index;
            animStep = 0; // Reset animation step for new index
            animPhase = 0.0; // Reset animation phase
            Serial.print("➡ New selected_index: ");
            Serial.println(lastIndex);
        }
    } else {
        Serial.print("✗ Firebase error: ");
        Serial.println(fbdo.errorReason());
    }
}

void animateLEDs() {
    unsigned long now = millis();
    if (now - lastAnimTime < 15) return; // Animation interval (66 FPS)
    lastAnimTime = now;

    // Clear all LEDs

```

```

for (int i = 0; i < NUM_LEDS_TOTAL; i++) {
    strip.SetPixelColor(i, RgbColor(0, 0, 0));
}

// Increment animation phase much faster
animPhase += 0.4;
if (animPhase > 6.28) animPhase = 0; // Reset at 2π

// --- BOX 1: (LEDs 0-19) for index 1 ---
if (lastIndex == 1) {
    fastRainbowChase(0, 20);
}
// --- BOX 2: (LEDs 20-39) for index 2 ---
else if (lastIndex == 2) {
    smoothBreathing(20, 40);
}
// --- BOX 3: (LEDs 40-59) for index 3 ---
else if (lastIndex == 3) {
    policeSiren(40, 60);
}
// --- BOX 4: (LEDs 60-79) for index 4 ---
else if (lastIndex == 4) {
    professionalLoading(60, 80);
}
// --- BOX 5: (LEDs 80-98) for index 8 ---
else if (lastIndex == 8) {
    snakeChase(80, 99);
}
// --- BOX 6: (LEDs 99-119) for index 7 ---
else if (lastIndex == 7) {
    colorWipe(99, 119);
}
// --- BOX 7: (LEDs 119-139) for index 6 ---
else if (lastIndex == 6) {
    dataStream(119, 139);
}
// --- BOX 8: (LEDs 139-159) for index 5 ---
else if (lastIndex == 5) {
    matrixRain(139, 159);
}
// --- BOX 9: (LEDs 159-179) for index 9 ---
else if (lastIndex == 9) {
    neonPulse(159, 179);
}
// --- BOX 10: (LEDs 179-199) for index 10 ---
else if (lastIndex == 10) {
    professionalSineWave(179, 199);
}
else if (lastIndex == 11) {
    particleFlow(199, 218);
}
}
strip.Show();
animStep += 2; //animation step increment
}

// 1.
void fastRainbowChase(int start, int end) {
    int chasePos = (animStep) % (end - start);
    for (int i = start; i < end; i++) {
        if ((i - start) == chasePos) {
            strip.SetPixelColor(i, RgbColor(255, 0, 0)); // Bright Red
        } else if ((i - start) == ((chasePos + 1) % (end - start))) {
            strip.SetPixelColor(i, RgbColor(255, 165, 0)); // Orange
        } else if ((i - start) == ((chasePos + 2) % (end - start))) {
            strip.SetPixelColor(i, RgbColor(255, 255, 0)); // Yellow
        } else if ((i - start) == ((chasePos + 3) % (end - start))) {
            strip.SetPixelColor(i, RgbColor(0, 255, 0)); // Green
        } else if ((i - start) == ((chasePos + 4) % (end - start))) {
            strip.SetPixelColor(i, RgbColor(0, 0, 255)); // Blue
        }
    }
}

// 2.
void smoothBreathing(int start, int end) {
    float breathIntensity = (sin(animPhase * 1.5) + 1.0) / 2.0;
    breathIntensity = breathIntensity * breathIntensity; // Smooth curve

    int brightness = (int)(breathIntensity * 255);
    RgbColor breathColor = RgbColor(0, brightness/3, brightness); // Cool blue

    for (int i = start; i < end; i++) {
        strip.SetPixelColor(i, breathColor);
    }
}
}

```

```

// 3.
void policeSiren(int start, int end) {
    int mid = start + (end - start) / 2;

    if ((animStep % 12) < 6) {
        // Red half
        for (int i = start; i < mid; i++) {
            strip.SetPixelColor(i, RgbColor(255, 0, 0));
        }
        // Blue half
        for (int i = mid; i < end; i++) {
            strip.SetPixelColor(i, RgbColor(0, 0, 255));
        }
    } else {
        // Blue half
        for (int i = start; i < mid; i++) {
            strip.SetPixelColor(i, RgbColor(0, 0, 255));
        }
        // Red half
        for (int i = mid; i < end; i++) {
            strip.SetPixelColor(i, RgbColor(255, 0, 0));
        }
    }
}

// 4.
void professionalLoading(int start, int end) {
    int boxSize = end - start;
    int loadProgress = (animStep / 3) % (boxSize + 10);

    for (int i = start; i < end; i++) {
        int pos = i - start;

        if (pos < loadProgress && pos < boxSize) {

            strip.SetPixelColor(i, RgbColor(0, 180, 0));
        } else if (pos == loadProgress && loadProgress < boxSize) {

            strip.SetPixelColor(i, RgbColor(255, 255, 255));
        } else if (pos < boxSize) {

            strip.SetPixelColor(i, RgbColor(20, 20, 20));
        }
    }
}

// 5.
void snakeChase(int start, int end) {
    int snakeLength = 5;
    int headPos = animStep % (end - start + snakeLength);

    for (int i = start; i < end; i++) {
        int distFromHead = headPos - (i - start);

        if (distFromHead >= 0 && distFromHead < snakeLength) {
            float brightness = (snakeLength - distFromHead) / (float)snakeLength;
            if (distFromHead == 0) {
                strip.SetPixelColor(i, RgbColor(255, 255, 0)); // Yellow head
            } else {
                strip.SetPixelColor(i, RgbColor(0, 255 * brightness, 0)); // Green body
            }
        }
    }
}

// 6.
void colorWipe(int start, int end) {
    int wipePos = (animStep / 2) % ((end - start) * 2);

    for (int i = start; i < end; i++) {
        if (wipePos < (end - start)) {
            // Forward wipe
            if ((i - start) <= wipePos) {
                strip.SetPixelColor(i, RgbColor(255, 100, 0)); // Orange
            }
        } else {
            // Reverse wipe
            if ((i - start) >= ((end - start) * 2 - wipePos - 1)) {
                strip.SetPixelColor(i, RgbColor(0, 255, 100)); // Green
            }
        }
    }
}

// 7.

```

```

void dataStream(int start, int end) {
    for (int i = start; i < end; i++) {
        int pos = i - start;

        float packet1 = sin((pos * 0.5) - (animPhase * 2.0));
        float packet2 = sin((pos * 0.8) - (animPhase * 1.5));

        if (packet1 > 0.7) {
            strip.SetPixelColor(i, RgbColor(0, 255, 255));
        } else if (packet2 > 0.8) {

            strip.SetPixelColor(i, RgbColor(255, 255, 255));
        } else if (packet1 > 0.3 || packet2 > 0.4) {

            strip.SetPixelColor(i, RgbColor(0, 50, 100));
        } else {

            strip.SetPixelColor(i, RgbColor(0, 10, 20));
        }
    }
}

// 8.
void matrixRain(int start, int end) {
    for (int i = start; i < end; i++) {
        int pos = i - start;

        // Create falling digital rain drops
        float rain1 = sin((pos * 0.4) - (animPhase * 3.0));
        float rain2 = sin((pos * 0.6) - (animPhase * 2.5));
        float rain3 = sin((pos * 0.8) - (animPhase * 2.0));

        if (rain1 > 0.8) {

            strip.SetPixelColor(i, RgbColor(100, 255, 100));
        } else if (rain1 > 0.5 || rain2 > 0.7) {

            strip.SetPixelColor(i, RgbColor(0, 200, 0));
        } else if (rain1 > 0.2 || rain2 > 0.3 || rain3 > 0.6) {

            strip.SetPixelColor(i, RgbColor(0, 100, 0));
        } else {

            strip.SetPixelColor(i, RgbColor(0, 20, 0));
        }
    }
}

// 9.
void neonPulse(int start, int end) {
    float intensity = (sin(animPhase * 2) + 1.0) / 2.0;
    intensity = intensity * intensity;

    RgbColor color1 = RgbColor(255 * intensity, 0, 255 * intensity); // Purple
    RgbColor color2 = RgbColor(255 * intensity, 100 * intensity, 255 * intensity); // Pink

    for (int i = start; i < end; i++) {
        if ((i - start) % 2 == 0) {
            strip.SetPixelColor(i, color1);
        } else {
            strip.SetPixelColor(i, color2);
        }
    }
}

// 10.
void professionalSineWave(int start, int end) {
    for (int i = start; i < end; i++) {
        int pos = i - start;

        float wave = sin((pos * 0.6) + animPhase) * 0.5 + 0.5;

        int red = (int)(wave * 80);
        int green = (int)(wave * 150);
        int blue = (int)(wave * 255);

        strip.SetPixelColor(i, RgbColor(red, green, blue));
    }
}

// 11.
void particleFlow(int start, int end) {

```

```

for (int i = start; i < end; i++) {
    int pos = i - start;

    float particle1 = sin((pos * 0.7) - (animPhase * 1.8));
    float particle2 = sin((pos * 0.9) - (animPhase * 1.3));

    if (particle1 > 0.8) {
        strip.SetPixelColor(i, RgbColor(255, 255, 255));
    } else if (particle2 > 0.7) {
        strip.SetPixelColor(i, RgbColor(255, 200, 100));
    } else if (particle1 > 0.4 || particle2 > 0.3) {
        strip.SetPixelColor(i, RgbColor(100, 80, 20));
    } else {
        strip.SetPixelColor(i, RgbColor(20, 15, 5));
    }
}
}

```

Table: Source Code for the main program

4 Implementation

4.1 Description

4.1.1 Firebase Realtime Database Setup

The first step towards the creation of the Cloud-Based Smart Object Locator was the implementation of the Firebase Realtime Database. The database served as the middle ground between the web interface and embedded ESP32 hardware. It was chosen for its easy integration into the React (frontend) and ESP32 (firmware) environment, as well as for its ability to process real-time update data.

A new Firebase project was started in the Firebase Console, and an instance of the Realtime Database was started in test mode to enable development. The database had three main nodes:

```

https://draft2-1742d-default-rtdb.asia-southeast1.firebaseio.com/
├── auth
├── components
└── selected_index: "7"

```

Fig : Root Node of the Database

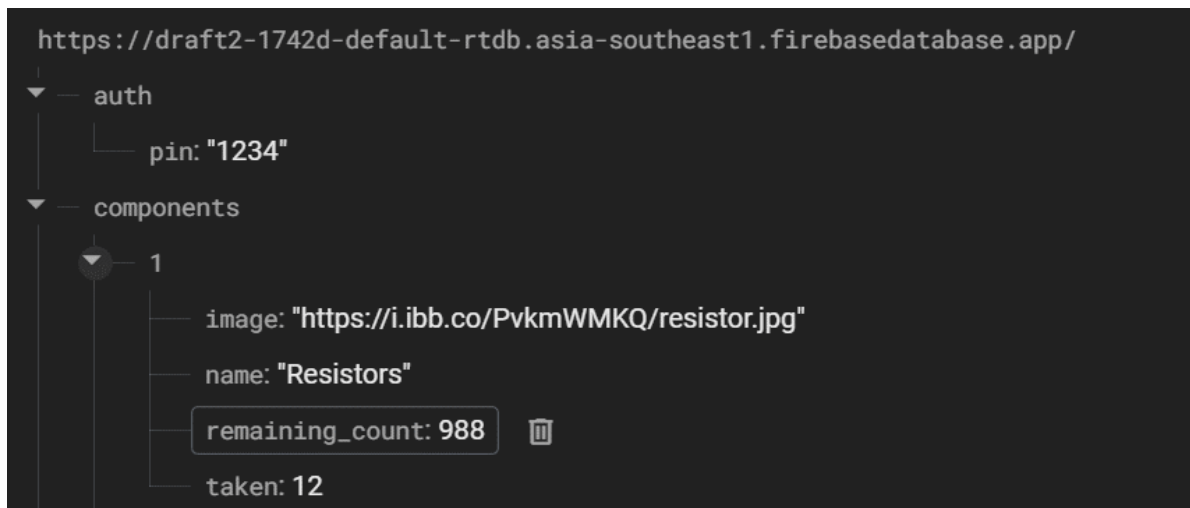


Fig: expanded view of component node

- **auth:** The node stored the login PIN for entering the website dashboard. It was an elementary security feature to prevent unwanted users.
- **components:** This node stored a list of all objects that were stored. Each record contained metadata such as:
 - **name:** The name of the object.
 - **image:** The URL of the object's image, which is hosted on a third-party website (imgbb.com).
 - **remaining:** A number representing how many of that object are left in stock.
 - **selected_index:** This important field sent the index of the object that was presently selected on the website to the ESP32. When the user clicked on an object on the dashboard, that respective index was updated here. The ESP32 also polled this node regularly in order to sense and respond to changes.

Tight read/write controls were put in place to provide safe communication and yet allow real-time updates. Tested, the system showed database updates with minimal latency, with effective and stable syncing between action from the user and reaction from the hardware.

4.1.2 Web Front-End Design with React

The second part of the system implementation was to develop a basic web interface to support remote access of the component locator system. The frontend of the system was built with React, a JavaScript library for developing responsive and dynamic web user interfaces. The interface was the primary interaction point of the user with the system and supported operations like selecting, searching, and editing component information.

The major attributes of this stage are described as below:

- **Login Page Functionality:** The application begins with a secure login page, where the user credentials are authenticated by comparing a PIN stored in the auth field of

the Firebase Realtime Database. The authentication process is implemented to protect the dashboard and system access to valid users only.



- **Dashboard Features:** Once successfully logged in, the user is taken to the dashboard page that provides full control over the object locator. The primary functionalities are:

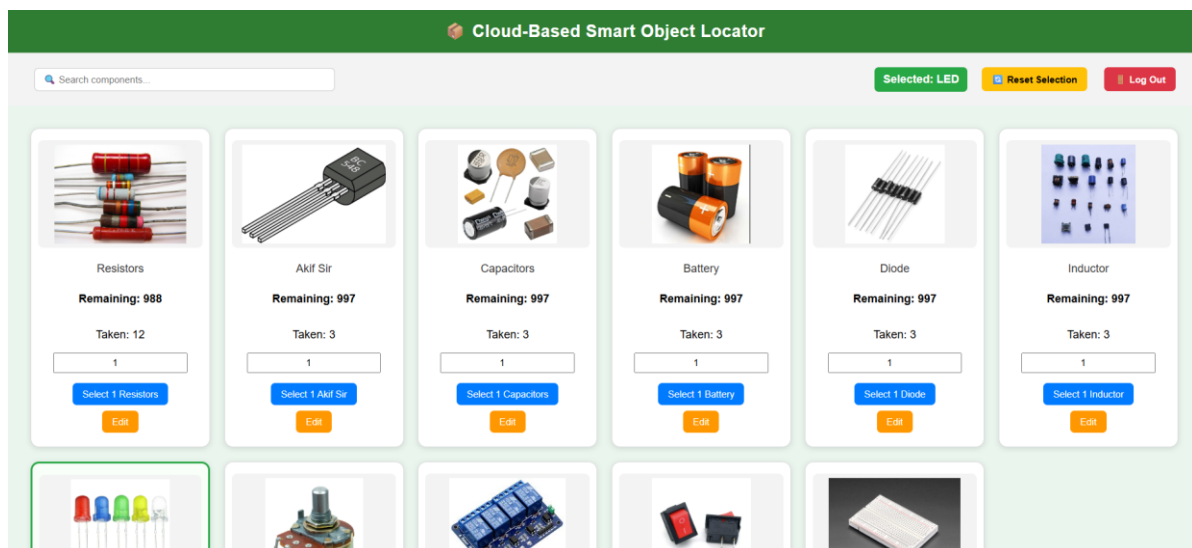


Fig: Dashboard

- **Component Selection:** Users can simply click on any of the given components to select it. It triggers an update of the selected_index field in Firebase that is read by the ESP32.

- **Component Management:** Users can add a new component, rename an existing component, or change the remaining number of a component.

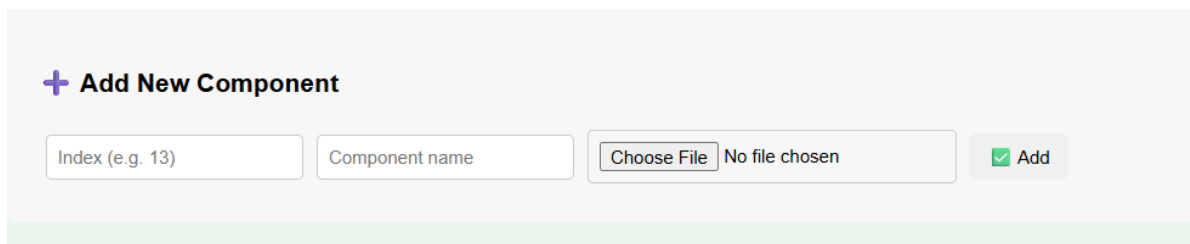


Fig: Adding New Component

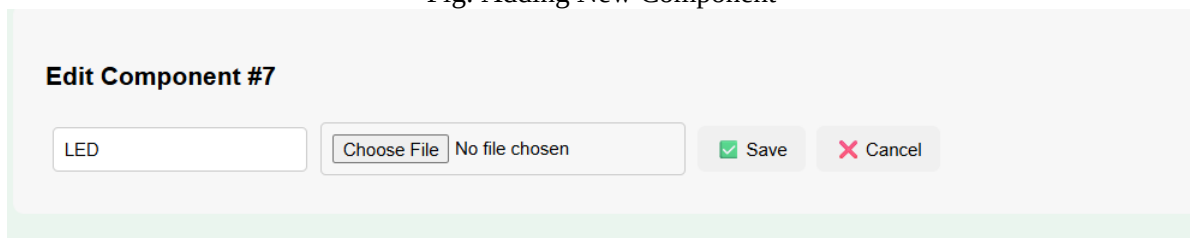
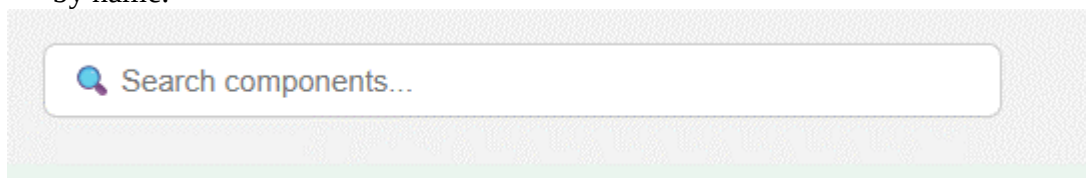
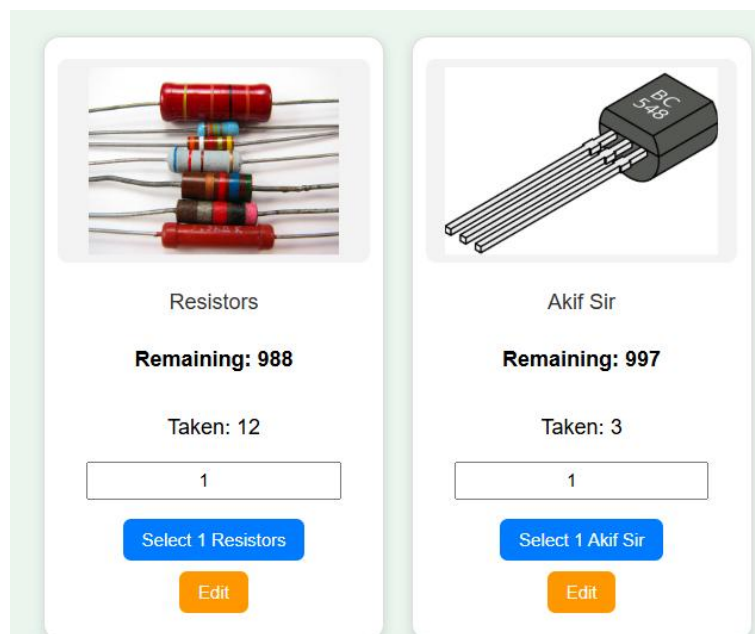


Fig : Edit Existing Component

- **Search and Filter:** A quick search box facilitates the quick search of individual parts by name.



- **Selection Status Display:** The currently selected item is displayed visually highlighted.

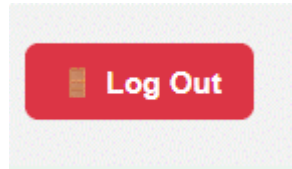


- **Reset & Auto Reset:** Users can reset the selection manually or have the system reset it automatically after a minute.



- **Logout Option:** A logout button provides the ability for the user to be logged off back to the login screen and close access.

-



- **Hosting and Deployment:** The React web application was initially developed and tested locally at localhost:3000. After being thoroughly tested, it was hosted online using Vercel, a cloud platform that provides seamless hosting for frontend projects at no cost.
- **Image Management:** Object images displayed on the dashboard were hosted at imgbb.com, which is an external image host. URLs of the images were stored in the Firebase node titled components alongside other metadata.

The frontend was developed to be lightweight, responsive, and easy to use, allowing for real-time user interaction with the ESP32 system. Through the integration of Firebase with React, two-way smooth communication was achieved to ensure that any user action on the web interface directly reflected in the physical system in an instant.

4.1.3 ESP32 Hardware Integration

The third step was integrating the ESP32 microcontroller with the necessary hardware components to create the physical object locator unit. The ESP32 was chosen because it has in-built Wi-Fi capabilities, dual-core processing capabilities, and the ability to support Firebase-based IoT solutions. It was used as the primary controller for processing data retrieved from the Firebase Realtime Database as well as controlling LED-based visual indicators.

Key hardware integration steps are as follows:

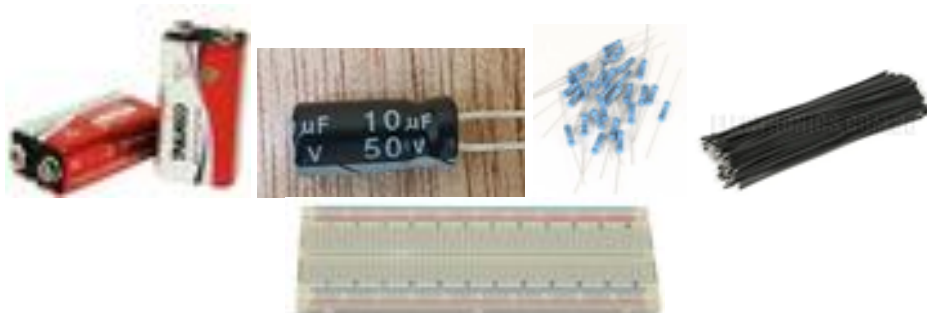
- **Microcontroller selection and configuration:** A development board ESP32 was selected since it possesses robust processing, wireless networking, and sufficient GPIO support. The board was compiled using the Arduino IDE, which offered necessary libraries for Firebase messaging, Wi-Fi networking, and WS2812B LED control.



- **LED Strip Configuration:** A WS2812B addressable LED strip was used to provide visual feedback by illuminating the chosen object's storage box. Each box was defined with a portion of the LED strip so that that specific segment could be illuminated or not. The strip was connected to an ESP32 GPIO pin and powered with the Adafruit_NeoPixelBus library.



- **Power Management:** For stable and safe operation, a regulated power supply of 9V–5V was employed. The device was powered by two 9V extra heavy-duty batteries for portability purposes. Decoupling capacitors 0.1uF and 10uF were applied to stabilize the voltage rail and reduce electrical noise. A 220-ohm resistor was placed between the ESP32 and the LED strip data input to prevent reflection of the signal and proper logic level matching.



- **Box and LED Layout:** The physical configuration was made up of plastic boxes arranged in a grid pattern. There were object associations for each box with a specific segment of the LED strip. If an object is selected via the web interface, the corresponding box lit up through a programmed LED animation.

This integration of hardware enabled the ESP32 to serve as a bridge between the physical world and the cloud, delivering real-time feeds, visual feedback, and smooth coordination between system components.

4.1.4 LED Animation Programming

In the fourth step of the implementation, we focused on developing attractive LED animation effects to visually highlight a storage box for a chosen object. The animations

formed the core function of the object locator system, allowing users to discover items easily and automatically through real-time visual feedback.

LED animations were programmed on the ESP32 using the NeoPixelBus library which supports smooth control of WS2812B addressable LEDs. Each box in the physical design was assigned a specific segment or index in the LED strip so that specific lighting and effects can be addressed.

Critical elements of this phase are described below:

- **Animation Trigger Logic:** The ESP32 periodically polled the value of `selected_index` from the Firebase Realtime Database. If it found a valid index, it triggered the animation of the corresponding box. This made the LED response real-time and appropriately linked to user input on the web interface.

```
// 7
void colorWipe(int start, int end) {
    int wipePos = (animStep / 2) % ((end - start) * 2);

    for (int i = start; i < end; i++) {
        if (wipePos < (end - start)) {
            // Forward wipe
            if ((i - start) <= wipePos) {
                strip.SetPixelColor(i, RgbColor(255, 100, 0)); // Orange
            }
        } else {
            // Reverse wipe
            if ((i - start) >= ((end - start) * 2 - wipePos - 1)) {
                strip.SetPixelColor(i, RgbColor(0, 255, 100)); // Green
            }
        }
    }
}
```

Fig: Example Animation Code Piece

Types of Animations Implemented:

Breathing Light: Soft fade-in and fade-out effect to gradually capture attention on the box.

Snake Chase: A flowing LED sequence going through the target segment, giving a sense of dynamic visual feedback.

Rainbow Cycle: A multicolored cycling rainbow pattern for easy identification.

Flash/Pulse: Ongoing on/off flashing for immediate alert-type feedback.

And many more



- **LED Mapping to Boxes:** Some LED indices were mapped into each physical box based on their location. The code did the mapping dynamically in order to facilitate flexible reconfiguring of boxes or animation regions.
- **Auto Reset Feature:** For power saving and avoidance of LED burnout, an auto-reset timer was implemented. The ESP32 automatically switched off the current LEDs and set the selected_index to a default value if selection of a new object had not occurred for 60 seconds.

This procedure ensured not only the efficient operation of the Cloud-Based Smart Object Locator but also a pleasing and intuitive visual experience. The animation logic added depth and interactive personality to the system, and what would normally be mere object selection became a smooth and intelligent physical response.

4.1.5 Firebase to ESP32 Communication

The fifth step was to establish smooth and real-time interaction between the ESP32 microcontroller and the Firebase Realtime Database. This step was critical to ensure that inputs provided by users through the web interface were being accurately and timely reflected in the physical LED system.

To allow for this, the ESP32 was set to connect to Firebase via the WiFiClientSecure and Firebase ESP32 library. The ESP32, on startup, had automatically connected to a pre-set Wi-Fi network and had logged into Firebase using a one-of-a-kind API key and the Firebase project database URL. These login details were hardcoded into firmware in order to allow the ESP32 to locate the specific instance of Firebase that it needed to talk to.

Principal functionalities of the communication step are elaborated below:

```
....  
WiFi connected!  
IP address: 192.168.0.107  
Firebase sign-up successful  
✅ Firebase initialized  
✅ OTA Ready!  
To update wirelessly:  
1. Go to Tools -> Port in Arduino IDE  
2. Select 'ESP32-Firebase-LED at 192.168.0.107'  
3. Upload your code wirelessly!  
✅ Setup complete! Firebase + OTA ready  
➡ New selected_index: 0  
➡ New selected_index: 7
```

Fig: Esp32 Serial Monitor

- **Firestore Initialization:** The ESP32 utilizes two significant identifiers to locate and access the appropriate Firestore project:
 - **API Key:** Authenticates the device to Firestore services.
 - **Database URL:** Directs the ESP32 to the exact real-time database location where auth, components, and selected_index data are housed.
- **Polling Mechanism:** Since Firestore Realtime Database lacks native support for push notification to ESP32, the polling method was adopted. The ESP32 was constantly monitoring (every few hundred milliseconds) the value of the selected_index node to determine whether any update was being sent from the web interface.

4.1.6 Physical Box Construction

The sixth step was the physical realization of the component storage system, which was the visual interpretation of the object locator interface. This step was to translate the logical output from the ESP32 and LED system into a structured, physical layout that would be easy for users to use in a lab or workshop setting.

The construction included creating a grid-based system of individual compartments (boxes), each assigned to a specific object in the Firestore database and can be selected via the web interface.

Main points of the physical construction are detailed below:



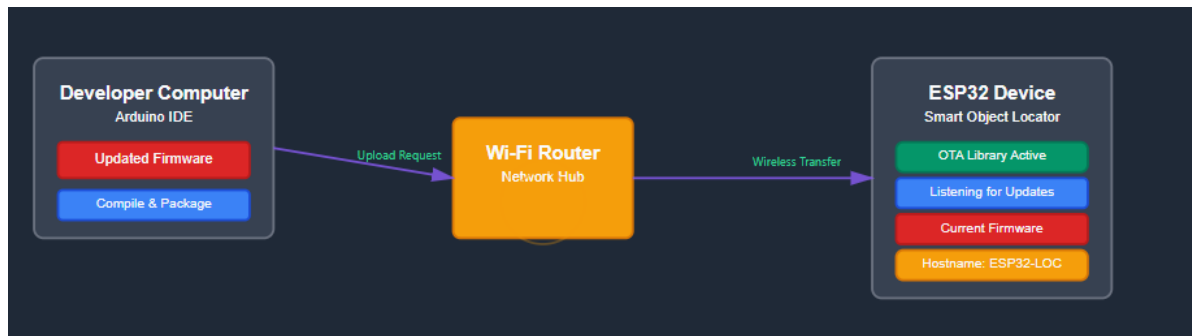
Fig: Physical Setup

- **Box Arrangement and Material:** A group of tiny plastic boxes was used to store different electronic components. The boxes were arranged in a rectangular grid to facilitate easier mapping with the LED index positions. A specific position identifier was assigned to each box that corresponded to the chosen_index from Firebase.
- **LED Integration per Box:** WS2812B addressable LED strip pieces were purposefully set in a manner that lit each compartment directly when its index was activated. The LED strip was carefully routed around the design to shade all boxes without overlap or ambiguity. Each box contains 20 LEDs.
- **Wiring and Connections:** LED strip was driven and regulated through a breadboard circuit interfaced with ESP32. Power rails were extended using jumper wires and flexible copper wires to minimize mess and enhance safety. All points of connection were verified for stability, particularly at LED segment joints.

4.1.7 OTA (Over-the-Air) Update Integration

The seventh step in the implementation was to activate Over-the-Air (OTA) update functionality for the ESP32 firmware. OTA updates allow the firmware on the microcontroller to be updated remotely, without the necessity of a physical USB connection or disassembling the system. This feature significantly improves the maintainability, scalability, and developer experience of the Smart Object Locator.

Key points of the process of integrating OTA are:



- **Why OTA:** During the process of development and testing, frequent firmware updates were needed for adding features, fixing bugs, and adjusting animations. Instead of needing to physically re-plug in the ESP32 into a computer each time, OTA allowed updates to be pushed directly over Wi-Fi, making the workflow easier and also enabling remote maintenance down the line.
- **OTA Implementation:**
 - Connecting the ESP32 to an already configured Wi-Fi network.
 - Assigning the ESP32 board with a recognizable hostname.
 - Initializing OTA services and registering callback functions to monitor update status (start, progress, end, and errors).
 - Running the OTA loop in the loop() function to listen for incoming update requests.
- **Update Process:**
 - After enabling OTA:
 - The ESP32 was visible as a network device in the Arduino IDE port list.
 - Firmware could be wirelessly uploaded directly from the IDE to the ESP32.
 - The board automatically booted into the new firmware after a successful upload.

5 Design Analysis and Evaluation

5.1 Novelty

This project introduces a novel integration of cloud-based control with real-time LED-based visual feedback to locate small objects in storage. Unlike traditional systems that rely on manual search or basic labeling, this system enables remote selection of storage boxes via a React-based GUI connected to Firebase Realtime Database. The selected box is highlighted with unique LED animations using WS2812B addressable LEDs controlled by ESP32. The design promotes efficient object retrieval and can be extended for industrial inventory or educational IoT labs.

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

The project is low-voltage and safe for lab environments. All electrical components are powered through regulated 5V–9V sources. The enclosure for LED hardware prevents direct contact with circuitry, ensuring safety during operation.

5.2.2 Considerations to environment

Component selection focused on low-power, reusable hardware. The ESP32 and WS2812B LEDs are energy-efficient and emit minimal heat. Modular construction allows future reuse of components, minimizing electronic waste.

5.2.3 Considerations to cultural and societal needs

In educational institutions and small workshops, time loss due to disorganization is common. This system addresses a culturally relevant need in Bangladesh for low-cost, smart lab solutions and encourages IoT adoption in learning environments.

5.3 Investigations (PO(d))

5.3.1 Design of Experiment

Experiments were designed to test response time from GUI to LED response, data integrity between Firebase and ESP32, and animation effectiveness in low-light vs. daylight conditions.

5.3.2 Results and Analysis

Average system latency from button click to LED illumination was ~300–500 ms, suitable for user experience. No packet loss was observed between Firebase and ESP32 during operation. When project is initialised, a bit of delay is recognized to illuminate the LEDs.

5.3.3 Interpretation and Conclusions on Data

The data confirms the system is reliable and fast enough for real-time application. The animations are distinguishable even in bright lighting, ensuring effective identification of the selected box.

5.4 Limitations of Tools (PO(e))

- **ESP32:** Limited GPIO when using many WS2812B LEDs can affect scalability.
- **Firestore Realtime DB:** Not optimized for high-throughput commands in free-tier; prone to minor delays under network fluctuations.
- **WS2812B LEDs:** Sensitive to precise timing; requires careful programming on ESP32.
- **GUI:** Depends on internet availability and browser compatibility.

5.5 Impact Assessment (PO(f))

5.5.1 Assessment of Societal and Cultural Issues

The system improves productivity in academic and maker environments, which aligns with societal goals for digital literacy and efficient learning spaces.

5.5.2 Assessment of Health and Safety Issues

No hazardous materials or high-voltage risks are present. The system is safe for users of all skill levels.

5.5.3 Assessment of Legal Issues

All software used (Firestore, React, Vercel) is either open-source or under fair-use licenses. No copyrighted content was used.

5.6 Sustainability Evaluation (PO(g))

This system promotes sustainability by reducing the need for printed labels or manuals and enabling reuse of components. Its modular design supports upgrades or repurposing of parts, extending lifecycle and reducing e-waste.

5.7 Ethical Issues (PO(h))

All team members contributed fairly, and no plagiarism or unauthorized code use was involved. The cloud services were used within their free-tier limits. Ethical handling of data and acknowledgment of all contributions were maintained throughout the project.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Name	Student ID	Key Contributions
Anas Rohan	2006064	- Developed the React-based GUI and integrated Firebase Realtime Database - Designed UI logic including security pin & logout - Deployed on Vercel
Afnan Ali	2006063	- Led hardware setup with ESP32 and WS2812B LEDs - Implemented animations (e.g., Snake Chase, Breathing Light) - Assembled circuit and box design
Md. Shakib Hossain	2006064	- Handled Firebase-ESP32 sync logic - Programmed ESP32 firmware logic and polling - Conducted debugging and performance testing
Al Amin Shawon	1806098	- Assisted in testing hardware and optimizing animations - Conducted component cost analysis and timeline planning - Helped with presentation preparation

6.2 Mode of TeamWork

Our team adopted a collaborative and distributed mode of teamwork. Work was divided based on individual strengths in software development, embedded systems, and documentation. Regular meetings were held (in-person and online) to:

- Review progress and align integration points (e.g., Firebase connection).
- Share debugging challenges and solutions.
- Reassign overlapping tasks when one member faced delays.

A shared version-controlled folder and GitHub repository ensured everyone could track progress and contribute asynchronously. The distributed approach encouraged both autonomy and accountability

6.3 Diversity Statement of Team

The team brought together members with diverse technical interests:

- Two members had a stronger background in software and cloud technologies.
- Two others focused more on embedded hardware and circuit implementation.

This skill diversity ensured balanced contribution across both the hardware and software

layers of the system. Despite different strengths, all members gained cross-functional exposure by rotating roles during integration and testing phases.

6.4 Logbook of Project Implementation

Date	Milestone Achieved	Individual Role	Team Role	Comments
May 18	Project idea finalized	All	Defined scope and toolchain	
June 03	Firebase DB and React frontend setup	Shakib, Anas	Team reviewed web interface	Basic GUI working
June 16	ESP32 LED wiring and coding started	Anas, Shawon	Team reviewed animation logic	Connected 1m LED strip
June 21	Firebase-ESP32 communication tested	Anas	Tested with dummy data	Success rate ~90%
July 08	Implemented animation effects	Afnan, Shakib	Tested animations on box	Rainbow, Snake, Breathing added
July 10	Vercel website hosted	Anas	Verified real-time response	Ready for demo
July 21	GUI Security PIN, logout, and index 0 logic	Afnan, Anas	UI-DB logic verified	Reset working fine
July 23	Final system test with full hardware	All	Full system tested end-to-end	Latency ~300–500 ms
July 27	Presentation and report preparation	All	Slides prepared and practice done	

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

The **Cloud-Based Smart Object Locator** is an embedded system integrated with cloud technology to help users locate specific objects stored in multiple compartments. Through a secure web interface, users can remotely trigger distinct LED animations on an ESP32-powered box to visually identify the desired location. This system bridges hardware and software using Firebase Realtime Database, React web technologies, and addressable WS2812B LEDs. It is ideal for use in electronics labs, workshops, and IoT-based learning environments. The design ensures minimal wiring, ease of deployment, and intuitive interaction, making it a smart, scalable, and educational solution.

7.2 Github Link

7.3 YouTube Link

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Materials

Component	Quantity	Unit Price (BDT)	Total (BDT)
ESP32 Dev Wifi+Bluetooth	1	450.00	450.00
Addressable LED Strip	6 (1 meter, 60 LEDs each)	600.00	3600.00
9V Battery	2	74.00	148.00
Capacitors	2 packs	25.00	50.00
Resistors	4 packs	10.00	40.00
Jumper wires	—	—	162.00
		Total	4450.00

8.2 Calculation of Per Unit Cost of Prototype

The prototype system was built with low-cost components readily available in local markets. The total cost of a single unit is **4,450 BDT**, which includes the controller, LEDs, basic passive components, and power supply.

This cost can be optimized further if the system is mass-produced or integrated on a custom PCB, removing the need for breadboards and jumper wires.

8.3 Timeline of Project Implementation

Phase	Duration	Timeline
Idea Finalization	1 week	May 11 – May 18
GUI Development & Firebase Setup	2 weeks	May 20 – Jun 03
Hardware Setup	2 weeks	Jun 05 – Jun 19

Firebase to ESP32 Communication	1 week	Jun 20 – Jun 27
LED Animation and Box Arrangement	1.5 weeks	Jul 6 – Jul 16
Deployment and Final Testing	1 weeks	Jul 16 – Jul 23
Report and Presentation Preparation	1 weeks	Jul 24 – Jul 31

9 Future Work (PO(I))

To further enhance the system's capability, reliability, and scalability, the following improvements are proposed:

- **1. Two-Way Communication with Feedback**
Currently, the system sends a one-way command from the website to the ESP32. In future versions, implement feedback from the ESP32 to the Firebase database (e.g., status confirmation, error reporting, battery status).
- **2. Mobile App Integration**
Develop a mobile application (Android/iOS) using Flutter or React Native. This would provide a more user-friendly and accessible interface beyond browser-only access.
- **3. Object Detection Sensors**
Integrate IR or ultrasonic sensors inside boxes to confirm if the object is present or removed.
This would make the system intelligent and capable of tracking real-time object presence.
- **4. Voice-Controlled Activation**
Use Google Assistant, Alexa, or voice APIs to locate items using voice commands (e.g., “Find screwdriver” → triggers box).
- **5. RFID or QR Code Labeling**
Attach RFID tags or QR codes to individual items and scan them using a phone or reader to automate item logging and retrieval.
- **6. Power Optimization**
Replace 9V batteries with rechargeable Li-ion batteries and include a charging circuit. Consider deep sleep modes for ESP32 to save power when idle.
- **7. Custom PCB Design**
Replace the breadboard circuit with a compact, well-routed custom PCB.
This would reduce wiring mess, size, and improve reliability.
- **8. Multi-User Authentication**
Enable multiple authorized users with different access levels (e.g., Admin, Lab Assistant, Student).
Store login data and log securely in Firebase.
- **9. Inventory Logging and History**
Add a Firebase-based log system to track when and how often certain items are accessed. Useful for lab supervisors or tool management.
- **10. Scalability for Industrial Applications**
Extend the system to manage dozens or hundreds of compartments in warehouses or factories. Include Wi-Fi mesh networks for distributed ESP32 nodes.

10 References

- ❑ Espressif Systems. (n.d.). *ESP32 Series Datasheet Version 4.9*.
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- ❑ Mouser. (n.d.). Addressable LEDs 101: Control, design, and implementation. In *WHITEPAPER*.
https://www.mouser.com/pdfDocs/Dialight_Whitepaper_Addressable_LEDs_101.pdf
- ❑ Pachipala, Y., CHKaza, L., Anupama, P., & Bhavani, K. L. (2020). Object Tracking System using Iot. *International Journal of Innovative Technology and Exploring Engineering*, 9(3), 1357–1362. <https://doi.org/10.35940/ijitee.b7467.019320>